



Spot and Merge: A Hybrid Context Biasing Approach for Rare Word and Out of Vocabulary Recognition

Jatin Agrawal, Bramhendra Koilakuntla, Srikanth Konjeti

¹Observe AI, India,

{jatin.agrawal, koilakuntla.bramhendra, srikanth.konjeti}@observe.ai

Abstract

Deep context biasing improves rare word recognition in automatic speech recognition (ASR) but often struggles with larger biasing lists. Performance on Out-of-Vocabulary (OOV) words remains limited as the ASR model must learn to generate unseen token sequences. These limitations become more pronounced in contact center applications with prevalent business-specific terminologies. Additionally, using an existing ASR model is challenging, as deep biasing requires full joint training of the ASR model and the biasing module. To address these issues, we introduce ‘*spot and merge*’ (SAM), a novel Low-Rank Adapter (LoRA) based system that spots bias words in the cross-attention weights of the biasing module and merges them with ASR output. Unlike existing methods, our approach maintains strong performance even with larger biasing lists, achieving a 1.0% absolute word error rate (WER) reduction on LibriSpeech. It also demonstrates robust OOV recognition on an in-house contact center dataset.

Index Terms: speech recognition, contextual-biasing

1. Introduction

Advancements in end-to-end (E2E) architectures for ASR have greatly improved transcription accuracy and scalability, making them essential for a range of applications, including contact center operations. Prominent E2E approaches such as [1–5] demonstrate exceptional performance. However, these models struggle to recognize rare and OOV words. It is especially critical in contact center settings, where customer interactions frequently involve domain-specific language, including brand names, product names and proper nouns.

Context-biasing (CB) methods aim to improve the recognition of rare words by using a predefined list of biasing words and phrases, known as the biasing list. This approach is particularly relevant in contact center applications, where brand names and product names—commonly forming the bulk of these words—are typically known beforehand. Research in this area can be broadly categorized into two categories. The first category relies on shallow fusion with an external language model and on-the-fly rescoring during decoding [6–9]. Some other methods in this category integrate E2E ASR models with weighted finite state transducers for context-biasing [10, 11]. Shallow fusion methods are highly effective as they preserve the performance of the base model and eliminate the need for retraining, ensuring adaptability with minimal overhead. However, such systems integrate context at inference and do not leverage model’s internal representations, leading to weaker context understanding.

The second category of context-biasing methods involves integrating context directly into ASR models known as deep biasing methods. Initial studies in this field established the founda-

tion [12, 13], leading to the development of the Contextual Adapter [14], which has since emerged as one of the most widely adopted methods for deep contextual biasing. It has inspired numerous derivative studies [15–22]. Specifically, works like [20, 21] have achieved promising results by embedding contextual information into the model’s internal representations, enabling richer context understanding.

However, the performance of these approaches degrades significantly as the size of the biasing list increases. Additionally, they don’t allow adapting an existing ASR model as they require joint training of the ASR model with the biasing module. Furthermore, their performance on OOV words remains limited as they rely on the ASR model to output unseen token sequences. To overcome these limitations, we propose SAM, a system that integrates the strengths of both shallow and deep fusion methods. Our novel ‘*spot and merge*’ algorithm leverages cross-attention weights to spot bias phrases thereby, reducing reliance on the ASR model. Instead of full training, we adapt our existing ASR model using LoRA [23]. Our approach shows superior performance for larger biasing lists ($N = 1000$), achieving 1.5% and 0.2% absolute WER reduction as compared to [20] and [21] respectively. We also demonstrate a 73% absolute improvement in OOV detection F1 score on an in-house contact center specific dataset. We introduce an attention-based loss that enhances performance. Furthermore, we show that Phoneme-BPE [24] outperforms Word-BPE for OOV recognition. Our key contributions are as follows:

1. We propose a novel ‘*spot and merge*’ strategy that leverages cross-attention weights of the biasing module. It remains effective even for large biasing lists
2. We show that jointly training LoRA based audio encoder is enough to train a robust context biasing module. Thus, adapting an existing ASR model instead of retraining.
3. We introduce a new attention-based auxiliary loss function designed to enhance cross-attention weights, leading to better alignment and improved context-biasing performance.
4. We highlight the effectiveness of using Phoneme-BPE instead of Word-BPE for the OOV detection task, achieving notable improvements in recognition accuracy.

2. Methodology

Our proposed solution (SAM) consists of three components i) Pretrained ASR Model ii) Context Adapter (CA) iii) Spot and Merge module. Figure 1 represents the overall workflow.

2.1. Pretrained ASR Model

The ASR model is a Conformer-CTC [25] based model that consists of an audio encoder of size 100M parameters, paired with a CTC [1] output layer. It is trained on a large annotated dataset of 80,000 hours of contact center recordings. Inference is performed using CTC decoding with beam-search.

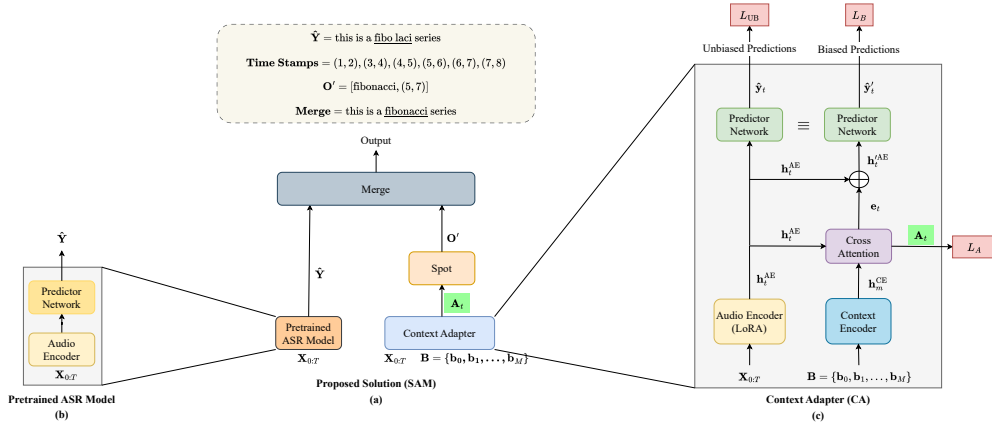


Figure 1: (a) Our Proposed solution with three major components i) Pretrained ASR Model ii) Context Adapter (CA) iii) Spot and Merge Module. (b) Pretrained ASR model consists of an audio encoder and a predictor network (c) CA module consists of a LoRA adapted audio encoder (One from the ASR Model), a context encoder, a cross attention module and a shared predictor network. Unbiased loss (L_{UB}), Bias loss (L_B) and Attention loss (L_A) are applied to train the context adapter module. The attention weights ($\mathbf{A}_t$) form the CA module are used to spot bias phrases ($\mathbf{O}'_n$). Spotted phrases are then merged with ASR outputs to produce the final transcript.

2.2. Context Adapter (CA)

Drawing inspiration from [17, 20, 21], the context adapter (CA) module is designed with three main components: i) Audio Encoder, ii) Context Encoder, and iii) Cross Attention

2.2.1. Audio Encoder

LoRA adapter is applied to the audio encoder of the pre-trained ASR model 2.1, denoted as $\text{AudioEnc}(\cdot)$. It processes an input acoustic feature sequence of length T , $\mathbf{X} = \{\mathbf{x}_t \in \mathbb{R}^D \mid t = 1, \dots, T\}$, and generates a high-level representation $\mathbf{H}^{\text{AE}} = \{\mathbf{h}_t^{\text{AE}} \in \mathbb{R}^S \mid t = 1, \dots, T'\}$, where $T' < T$. This operation is expressed as: $\mathbf{h}_t^{\text{AE}} = \text{AudioEnc}(\mathbf{x}_t)$.

Here, $\mathbf{x}_t$ and $\mathbf{h}_t^{\text{AE}}$ represent D -dimensional and S -dimensional acoustic features at frame t , respectively. A predictor network, denoted as $\text{Pred}(\cdot)$, processes $\mathbf{h}_t^{\text{AE}}$ to produce logits $\hat{\mathbf{Y}} = \{\hat{\mathbf{y}}_t \in \mathbb{R}^V \mid t = 1, \dots, T'\}$ over a vocabulary V . For a ground truth sequence of length U , $\mathbf{Y} = \{\mathbf{y}_u \in \mathbb{R}^V \mid u = 1, \dots, U\}$, the unbiased loss L_{UB} :

$$L_{UB} = \text{CTC}(\mathbf{Y}, \hat{\mathbf{Y}}), \quad (1)$$

2.2.2. Context Encoder

The Context Encoder encodes bias phrases in a fixed size embedding vector. $\mathbf{B} = \{\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_M\}$, represents a M size list of bias phrases. Each bias phrase $\mathbf{b}_m$, with $m \in [0, M]$, is processed by the context encoder employing a BiLSTM model. The first phrase, $\mathbf{b}_0$, is always a $\langle \text{nobias} \rangle$ token to handle cases where the bias list may not be relevant. The context encoder encodes each bias phrase into a fixed-size, S -dimensional phrase embedding vector, represented as $\mathbf{H}^{\text{CE}} = \{\mathbf{h}_m^{\text{CE}} \in \mathbb{R}^S \mid m = 0, \dots, M\}$, where: $\mathbf{h}_m^{\text{CE}} = \text{BiLSTM}(\mathbf{b}_m)$.

2.2.3. Cross Attention

Given a list of phrase embeddings $\mathbf{H}^{\text{CE}}$ and audio features $\mathbf{H}^{\text{AE}}$, multi-headed attention (MHA) style single attention head is employed to align phrases with audio frames. Specifically, projection matrices $\mathbf{W}_K, \mathbf{W}_Q \in \mathbb{R}^{S \times S}$ are utilized for keys and queries, respectively. The attention weights $\mathbf{A}_t$ (figure. 2) at time t are computed as:

$$\mathbf{A}_t = \text{Softmax} \left(\frac{\mathbf{h}_t^{\text{AE}} \mathbf{W}_Q (\mathbf{H}^{\text{CE}} \mathbf{W}_K)^{\top}}{\sqrt{S}} \right), \quad (2)$$

The biasing vector $\mathbf{e}_t$ is then obtained as: $\mathbf{e}_t = \mathbf{A}_t \mathbf{H}^{\text{CE}} \mathbf{W}_V$, where $\mathbf{W}_V \in \mathbb{R}^{S \times S}$ is the value projection matrix. The

context-aware encoder representation $\mathbf{h}_t^{\text{AE}}$ is computed by applying element-wise addition between $\mathbf{h}_t^{\text{AE}}$ and $\mathbf{e}_t$, as follows:

$$\mathbf{h}_t^{\text{AE}} = \mathbf{h}_t^{\text{AE}} \oplus \mathbf{e}_t. \quad (3)$$

2.2.4. Bias Loss (L_B)

The context-aware encoder representation $\mathbf{h}_t^{\text{AE}}$ (eq. 3) is passed to the prediction network $\text{Pred}(\cdot)$ to produce biased outputs $\hat{\mathbf{y}}'_t$: $\hat{\mathbf{y}}'_t = \text{Pred}(\mathbf{h}_t^{\text{AE}})$. The target biased output sequence $\mathbf{Y}'$ is created from $\mathbf{Y}$ by retaining only the phrases present in $\mathbf{B}$ and masking the remainder with the $\langle \text{nobias} \rangle$ tokens. For instance,

$$\begin{aligned} \mathbf{Y} &= \text{this is a fibonacci series} \\ \mathbf{Y}' &= \# \# \# \text{ fibonacci } \# \end{aligned} \quad (4)$$

$\#$ represents $\langle \text{nobias} \rangle$ token. The CTC loss is then applied between $\hat{\mathbf{Y}}'$ and $\mathbf{Y}'$, referred to as the bias loss (L_B):

$$L_B = \text{CTC}(\hat{\mathbf{Y}}', \mathbf{Y}'). \quad (5)$$

2.2.5. Attention Loss (L_A)



Figure 2: Attention Matrix ($\mathbf{A}$). The x-axis represents time (t) and y-axis represents phrases (m). Lighter color corresponds to higher probability.

Several studies have sought to enhance the quality of the attention weights generated by the CA module by incorporating auxiliary losses [15, 21]. We use Guided Attention CTC (GA-CTC) loss proposed in [21] since it is most effective. First, we obtain the list of bias phrases present in a given input $\mathbf{C} = (\mathbf{c}_1, \dots, \mathbf{c}_L)$, where $\mathbf{c} \in [1, M]$, excluding the $\langle \text{nobias} \rangle$ token. Here, L represents the number of bias phrases present in a given input including repetition. In the attention matrix $\mathbf{A} \in \mathbb{R}^{T' \times (M+1)}$, each element $\mathbf{A}_{tm}$ represents the likelihood of observing the phrase with index m at time t as seen in the figure 2. The GA-CTC loss is then computed using $\mathbf{C}$ as the label as:

$$L_{\text{GA-CTC}} = \text{CTC}(\mathbf{A}, \mathbf{C}). \quad (6)$$

However, given the loose alignment nature of CTC, GA-CTC loss is difficult to train [21]. It is particularly unstable in the

early stages of training. To overcome this, we introduce an additional, Phrase Spot (PS) loss (L_{PS}). This loss reduces the weight of absent phrases, allowing the model to focus only on the phrases present in a given input. First we obtain $\mathbf{K} = (\mathbf{k}_1, \dots, \mathbf{k}_M)$, where $\mathbf{k}_m \in \{0, 1\}$ that denotes absence or presence of a phrase (excluding $\langle \text{nobias} \rangle$) in a given input. Next, we generate the likelihood $\mathcal{P}_m$ of spotting the phrase (m) for a given input using attention matrix $\mathbf{A}$.

$$\mathcal{P}_m = \tanh \left(\sum_{t=0}^{T'} \mathbf{A}_{tm} \right) \quad (7)$$

$\tanh(\cdot)$ keeps $\mathcal{P}_m$ in $[0, 1]$ range as $\sum_{t=0}^{T'} \mathbf{A}_{tm}$ ranges from $[0, \infty]$. Finally, CrossEntropy($\cdot$) loss is applied between $\mathbf{K}$ and $\mathcal{P}$ to obtain L_{PS} :

$$L_{PS} = \text{CrossEntropy}(\mathcal{P}, \mathbf{K}) \quad (8)$$

The attention loss L_A now becomes:

$$L_A = L_{GA-CTC} + L_{PS} \quad (9)$$

Similarly, the overall objective function to train the CA module from eq 1, 5, 9, becomes: $L = L_{UB} + L_B + L_A$.

2.3. Spot and Merge

For inference, existing deep biasing methods [12–15, 20, 21] utilize the ASR model jointly trained with the biasing module. This hinders their performance on extremely rare and OOV words since the model struggles to produce unseen token sequences correctly. We present our novel ‘spot and merge’ approach that eliminates the need for the ASR model to produce these token sequences. Instead, it leverages attention matrix ($\mathbf{A}$) generated by the CA module to spot these phrases and merges them with the ASR model output. As outlined in Algorithm 1, our approach is as follows:

Each element $\mathbf{A}_{tm}$ in $\mathbf{A}$ represents the likelihood of observing a phrase with index m at time t . The phrase index with the maximum likelihood at each timestamp t is denoted as $\mathbf{p}_t$:

$$\mathbf{p}_t = \text{argmax}_{m \in [0, M]} (\mathbf{A}_t) \quad (10)$$

A forward pass from $t = 0$ to $t = T'$ can calculate start and end timestamps for each spotted phrase, marking the end when a streak is broken. A phrase can be spotted multiple times in different parts of the utterance, represented as a list of start and end timestamps called time-bounds $\mathbf{O}_m$.

$$\mathbf{O}_m = \{(l_i, r_i) \mid i \in \mathbb{R} \geq 0\}, \quad \text{where } m \in [0, M] \quad (11)$$

$\mathbf{O}_m$ is empty for unspotted phrases, and $\mathbf{O}_0$ (representing $\langle \text{nobias} \rangle$) is omitted. In cases where multiple phrases in the bias list have similar pronunciation, the end timestamp of one phrase can be adjacent to the start timestamp of another phrase, i.e., $\text{EndTime}(\mathbf{O}_a) + 1 = \text{StartTime}(\mathbf{O}_b)$, where $a, b \in [1, M]$. For example, terms *fibonacci* and *fibopathy* share the prefix ‘*fib*’, and can lead to adjacent time bounds. However, only one of these is the correct prediction.

To address this, we retain the phrase with the longest bound and filter out the rest of the adjacent phrases. Additionally, we update its time bound as the union of all adjacent bounds. This operation is denoted by $\text{FilterAdj}(\cdot)$:

$$\mathbf{O}' = \text{FilterAdj}(\mathbf{O}), \quad \mathbf{O}' = \{\mathbf{O}'_m \mid m = 1, \dots, M\} \quad (12)$$

The merge module combines ASR model output $\hat{\mathbf{Y}}$ with phrase spotter output $\mathbf{O}'$. It identifies overlapping words in $\hat{\mathbf{Y}}$ as per timestamps and replaces them with phrases. To reduce false positives, following [9] only words with word-confidence less than w_{th} are replaced. This shallow fusion approach ensures compatibility with any ASR model generating reliable timestamps and word-level confidence scores.

Algorithm 1 Spot and Merge Algorithm

Require: Audio frames $\mathbf{X}$, Bias phrases $\mathbf{B}$, ASR output $\hat{\mathbf{Y}}$, Word Confidence threshold w_{th} .

Output: Final text with merged phrases.

```

1:  $\mathbf{A} = \text{CA}(\mathbf{X}, \mathbf{B})$   $\triangleright$  Context Adapter attention weights
2:  $\mathbf{p} = \{\}$   $\triangleright$  Spotted Phrase index at each timestamp
3:  $\mathbf{O} = \{\}$   $\triangleright$  Initial time bounds for phrases
4:  $\mathbf{O}' = \{\}$   $\triangleright$  Filtered time bounds
5: for  $t = 0$  to  $T'$  do
6:    $\mathbf{p}_t = \text{argmax}_{m \in [0, M]} (\mathbf{A}_t)$ 
7:   Update time bounds  $\mathbf{O}_{\mathbf{p}_t}$  based on continuity of  $\mathbf{p}_t$ 
8: end for
9: for each phrase  $m \in [1, M]$  do
10:    $\mathbf{O}'_m = \text{FilterAdj}(\mathbf{O}_m)$   $\triangleright$  Filter adjacent bounds
11: end for
12: Final text =  $\hat{\mathbf{Y}}$ 
13: for each time bound  $\mathbf{O}'_m \in \mathbf{O}'$  do
14:    $\mathbf{W}_m = \{\hat{\mathbf{y}}_t \mid t \in \text{Overlap}(\mathbf{O}'_m, \hat{\mathbf{Y}})\}$ 
15:   if  $\text{Conf}(\mathbf{W}_m) < w_{th}$  then
16:     Replace  $\mathbf{W}_m$  in Final text with  $\mathbf{B}_m$ 
17:   end if
18: end for
19: return Final text
```

3. Experimental Setup

3.1. Training and Testing Datasets

We evaluate our system’s performance on LibriSpeech [26] and a contact-center specific, in-house conversational dataset.

3.1.1. In-house contact center dataset

In-house training dataset consists of 780 hours of annotated recordings from contact centers, featuring interactions between customers and agents in English. A list of rare words is generated by excluding the top 5000 most frequent words [11, 21]. During training, the bias list consists of rare words present in the utterances of the current batch along with a set of distractor phrases, which are randomly selected from the rare word list. Dynamically generating the bias list with variable distractors helps prevent model from overfitting.

Test Set 1: The bias list consists of only proper nouns and OOV terms. To evaluate false positives, the dataset maintains a 1:4 ratio of utterances containing at least one bias phrase to those containing no bias phrases. The testing dataset comprises 7900 utterances, totaling 19 hours of recordings.

Test Set 2: The bias list consists of only OOV words neither seen by the pretrained ASR model nor by CA module in its bias list. This dataset maintains a 1:9 ratio of utterances containing at least one bias phrase to those containing none. It consists of 9000 utterances in total, which amounts to 20 hours in total.

3.1.2. LibriSpeech Dataset

The LibriSpeech dataset contains of 960 Hours of training dataset with ‘*test-clean*’ and ‘*test-other*’ being the two test sets. The bias lists for training and testing datasets are prepared following the same protocols outlined in 3.1.1. For LibriSpeech experiments, the pretraining of the ASR model and training of the CA module is done on this LibriSpeech training dataset.

3.2. Modeling Details and Evaluation Metrics

The input audio features are 80-dimensional mel filter banks. The audio encoder, a LoRA-adapted model with rank 8, keeps only 1% of parameters trainable and outputs 512-dimensional features. Low-rank matrices are added to the attention layer $\{W_q, W_k, W_v\}$. For text, we use a SentencePiece BPE tokenizer [27] with vocabulary size of 1000. The context encoder

Table 1: Results on test-clean and test-other dataset of LibriSpeech. ‘No-Bias’ represents model without contextual biasing. N represents number of distractors. WER(U/B) denotes WER, U-WER/B-WER.

Model	test-clean								test-other							
	No-Bias	$N=0$		$N=100$		$N=1000$			No-Bias	$N=0$		$N=100$		$N=1000$		
	WER (U/B)	WER (U/B)	B-WER Δ	WER (U/B)	B-WER Δ	WER (U/B)	B-WER Δ		WER (U/B)	WER (U/B)	B-WER Δ	WER (U/B)	B-WER Δ	WER (U/B)	B-WER Δ	
P1	3.7 (2.2/15.8)	2.9 (1.6/13.0)	-18%	2.3 (1.5/8.6)	-46%	2.8 (1.7/12.4)	-22%		8.6 (5.9/32.0)	7.2 (4.8/28.0)	-13%	6.4 (4.7/21.2)	-34%	7.3 (5.0/27.9)	-13%	
P2	2.8 (1.9/10.2)	2.2 (1.8/5.3)	-48%	2.2 (1.8/5.1)	-50%	2.4 (1.9/6.4)	-38%		6.6 (4.9/22.0)	5.5 (4.7/12.4)	-44%	5.4 (4.7/12.2)	-45%	6.0 (5.0/15.3)	-31%	
SAM (Prop.)	2.8 (2.0/10.0)	2.3 (2.0/4.6)	-54%	2.2 (2.0/4.4)	-56%	2.3 (2.0/4.7)	-53%		6.8 (4.8/24.8)	5.6 (4.8/13.2)	-47%	5.6 (4.8/12.8)	-48%	5.8 (5.0/13.4)	-46%	

consists of a 128-dimensional embedding layer, a two-layer BiLSTM (128 units), and a linear projection layer. The Cross-Attention module applies a single attention head with a 512-dimensional embedding, followed by two linear layers to align dimensions. The predictor network uses a single layer mapping 512-dimensional vectors to the vocabulary size. During training, 50 distractors are added. The model is trained for 20 epochs with Adam optimizer, and a learning rate of 0.001.

We report results on in-house and LibriSpeech test sets. We report three standard word error rate (WER) metrics [11, 15]: i) WER (overall), ii) U-WER (Unbiased-WER): for words not in the bias list, and iii) B-WER (Biased-WER): for words in the bias list. Additionally, we compute Precision (P), Recall (R), and F1 scores for bias phrase detection.

Table 2: Results on our In-house Test Set 1.

Model	$N = 100$		$N = 500$	
	WER (U/B)	F1 (P/R)	WER (U/B)	F1 (P/R)
Pretrained ASR	17.3 (17.0/21.0)	85.9 (99.0/75.8)	17.3 (17.0/21.0)	85.9 (99.0/75.8)
SAM (V0)	17.3 (17.0/19.9)	89.1 (98.4/81.4)	17.3 (17.0/20.0)	87.2 (94.4/80.9)
SAM (Proposed)	17.2 (16.9/19.7)	89.2 (97.0/82.5)	17.2 (16.9/19.9)	87.3 (94.0/81.5)

4. Results and Discussion

Results on LibriSpeech. In Table 1, we present a comparative analysis of our proposed solution against existing CB methods, specifically P1 [20] and P2 [21], using the LibriSpeech dataset. Scores for P1 and P2 are taken from their respective papers. The *No-Bias* column in Table 1 reflects the performance of the base ASR model without contextual biasing. As illustrated, our proposed solution achieves the highest reduction in B-WER for any number of distractors. Noticeably, the performance gap between SAM, P1 and P2 widens as number of distractors increases. SAM achieves 53%, 46% B-WER reduction as compared to 22%, 13% by P1 and 38%, 31% by P2 on *test-clean* and *test-other* respectively for $N = 1000$. Furthermore, SAM yields the best overall WER on both *test-clean* and *test-other* dataset with large biasing list ($N = 1000$). While P2 achieves a better overall WER at $N = 0$, SAM achieves higher B-WER reduction. Moreover, $N = 0$ condition is irrelevant for real-world scenarios and thus considered not significant.

Noticeably, both P1 and P2 reduce U-WER as a result of contextual biasing while SAM does not. This observation can be attributed to our shallow fusion approach, which preserves the performance of the base ASR model on words not in biasing list. Interestingly, SAM exhibits a higher B-WER at $N = 0$ compared to $N = 100$. This discrepancy can be due to the fact that it is trained with $N = 50$ distractors, and the phrase-spotting algorithm relies on generating a probability distribution over the bias list. For $N = 0$, the quality of this distribution deteriorates, as it is effectively reduced to a binary choice between

the bias phrase and the `<nobias>` term. This impacts model’s context biasing ability, thereby increasing B-WER.

Results on In-house Test Set 1. This is a real-world conversational dataset hence is significantly more challenging than LibriSpeech. In Table 2 we showcase the effectiveness of our system with increasing number of distractors. SAM consistently reduces B-WER while increasing the Recall and F1-score. We also investigate the effectiveness of our proposed L_{PS} loss in Table 2. SAM(V0) represents a version of our model trained without L_{PS} , i.e., $L_A = L_{GA-CTC}$. In particular, SAM (proposed), trained with $L_{GA-CTC} + L_{PS}$ reported better recall and reduced B-WER. During training, it converged much faster than V0.

Table 3: Results on In-house Test Set 2 on OOV detection task.

Model	Precision	Recall	F1
Pretrained ASR	-	-	-
SAM	62.8	63.6	63.2
SAM +Phoneme-BPE	72.4	77.0	74.6

Results on In-house Test Set 2. This set contains only OOV terms in the bias list, carefully curated to ensure genuine OOV inclusion. Table 3 presents term-level precision, recall, and F1 scores. The pretrained ASR model fails to recognize any OOV term accurately. This issue primarily arises from spelling errors caused by inaccuracies in the model’s token sequence output. In contrast, SAM achieves a 63.6% improvement in F1 score leveraging similarities in audio-text embedding instead of generating token sequence.

A major challenge in our OOV recognition solution is ensuring that the context encoder generates accurate text embeddings, as it may struggle to infer pronunciations and produce unreliable representations. We hypothesize that phoneme-based tokenization can mitigate this issue, as phonemes inherently capture pronunciation, leading to improved context embeddings. To validate this, we replace Word-BPE with Phoneme-BPE, by converting text into phoneme sequences using a Grapheme-to-Phoneme model [28]. Each phoneme is assigned a unique Unicode character to apply Byte Pair Encoding. As shown in Table 3, the Phoneme-BPE model achieves an absolute 11.4% improvement in the F1 score over the Word-BPE model, supporting our hypothesis that phoneme-based tokenization enhances OOV recognition.

5. Conclusion and Future work

In this paper, we propose SAM, a novel hybrid context-biasing system that is highly effective and scalable to large biasing lists. It outperforms existing context-biasing techniques on the LibriSpeech dataset, and achieves strong results on a more challenging in-house dataset. We also introduce an auxiliary loss function based on attention weights, that improves performance. Furthermore, we demonstrate the effectiveness of Phoneme-BPE for OOV detection. Future work will involve extending the system’s multilingual capabilities, particularly for low-resource languages.

6. References

- [1] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” vol. 2006, 01 2006, pp. 369–376.
- [2] A. Graves and N. Jaitly, “Towards end-to-end speech recognition with recurrent neural networks,” in *Proceedings of the 31st International Conference on International Conference on Machine Learning - Volume 32*, ser. ICML’14. JMLR.org, 2014, p. II–1764–II–1772.
- [3] Y. He, T. N. Sainath, R. Prabhavalkar, I. McGraw, R. Alvarez, D. Zhao, D. Rybach, A. Kannan, Y. Wu, R. Pang, Q. Liang, D. Bhatia, Y. Shangguan, B. Li, G. Pundak, K. C. Sim, T. Bagby, S.-y. Chang, K. Rao, and A. Gruenstein, “Streaming end-to-end speech recognition for mobile devices,” in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 6381–6385.
- [4] J. Chorowski, D. Bahdanau, D. Serdyuk, K. Cho, and Y. Bengio, “Attention-based models for speech recognition,” in *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1*, ser. NIPS’15. Cambridge, MA, USA: MIT Press, 2015, p. 577–585.
- [5] W. Chan, N. Jaitly, Q. Le, and O. Vinyals, “Listen, attend and spell: A neural network for large vocabulary conversational speech recognition,” in *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2016, pp. 4960–4964.
- [6] I. Williams, A. Kannan, P. S. Aleksic, D. Rybach, and T. N. Sainath, “Contextual speech recognition in end-to-end neural network systems using beam search,” in *Interspeech*, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:52191719>
- [7] D. Le, G. Keren, J. Chan, J. Mahadeokar, C. Fuegen, and M. L. Seltzer, “Deep shallow fusion for rnn-t personalization,” in *2021 IEEE Spoken Language Technology Workshop (SLT)*, 2021, pp. 251–257.
- [8] D. Zhao, T. Sainath, D. Rybach, P. Rondon, D. Bhatia, B. Li, and R. Pang, “Shallow-fusion end-to-end contextual biasing,” 09 2019, pp. 1418–1422.
- [9] A. Andrusenko, A. Laptev, V. Bataev, V. Lavrukhin, and B. Ginsburg, “Fast context-biasing for ctc and transducer asr models with ctc-based word spotter,” 06 2024.
- [10] R. Huang, O. Abdel-Hamid, X. Li, and G. Evermann, “Class lm and word mapping for contextual biasing in end-to-end asr,” 07 2020.
- [11] D. Le, M. Jain, G. Keren, S. Kim, Y. Shi, J. Mahadeokar, J. Chan, Y. Shangguan, C. Fuegen, O. Kalinli, Y. Saraf, and M. L. Seltzer, “Contextualized streaming end-to-end speech recognition with trie-based deep biasing and shallow fusion,” *ArXiv*, vol. abs/2104.02194, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:233033913>
- [12] G. Pundak, T. N. Sainath, R. Prabhavalkar, A. Kannan, and D. Zhao, “Deep context: End-to-end contextual speech recognition,” in *2018 IEEE Spoken Language Technology Workshop (SLT)*, 2018, pp. 418–425.
- [13] M. Jain, G. Keren, J. Mahadeokar, and Y. Saraf, “Contextual rnn-t for open domain asr,” 06 2020.
- [14] K. M. Sathyendra, T. Muniyappa, F.-J. Chang, J. Liu, J. Su, G. P. Strimel, A. Mouchtaris, and S. Kunzmann, “Contextual adapters for personalized speech recognition in neural transducers,” in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 8537–8541.
- [15] D. Kulshreshtha, S. Dingliwal, B. Houston, and S. Bodapati, “Multilingual contextual adapters to improve custom word recognition in low-resource languages,” in *Proceedings of INTERSPEECH 2023*, 2023, pp. 3302–3306.
- [16] A. Alexandridis, K. M. Sathyendra, G. P. Strimel, F.-J. Chang, A. Rastrow, N. Susanj, and A. Mouchtaris, “Gated contextual adapters for selective contextual biasing in neural transducers,” in *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/10095322/>
- [17] K. Huang, A. Zhang, Z. Yang, P. Guo, B. Mu, T. Xu, and L. Xie, “Contextualized end-to-end speech recognition with contextual phrase prediction network,” in *Proceedings of INTERSPEECH 2023*, 2023, pp. 4933–4937. [Online]. Available: https://www.isca-archive.org/interspeech_2023/huang23d_interspeech.html
- [18] T. Munkhdalai, K. C. Sim, A. Chandorkar, F. Gao, M. Chua, T. Strohmaier, and F. Beaufays, “Fast contextual adaptation with neural associative memory for on-device personalized speech recognition,” in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 6632–6636. [Online]. Available: <https://ieeexplore.ieee.org/document/9747255/>
- [19] X. Fu, K. M. Sathyendra, A. Gandhe, J. Liu, G. P. Strimel, R. McGowan, and A. Mouchtaris, “Robust acoustic and semantic contextual biasing in neural transducers for speech recognition,” in *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/10095323/>
- [20] M. Shakeel, Y. Sudo, Y. Peng, and S. Watanabe, “Contextualized end-to-end automatic speech recognition with intermediate biasing loss,” 09 2024, pp. 3909–3913.
- [21] J. Tang, K. Kim, S. Shon, F. Wu, and P. Sridhar, “Improving asr contextual biasing with guided attention,” 04 2024, pp. 12 096–12 100.
- [22] F.-J. Chang, J. Liu, M. Radfar, A. Mouchtaris, M. Omologo, A. Rastrow, and S. Kunzmann, “Context-aware transformer transducer for speech recognition,” 11 2021.
- [23] E. J. Hu, yelong shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [24] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, K. Erk and N. A. Smith, Eds. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 1715–1725. [Online]. Available: <https://aclanthology.org/P16-1162/>
- [25] A. Gulati, C.-C. Chiu, J. Qin, J. Yu, N. Parmar, R. Pang, S. Wang, W. Han, Y. Wu, Y. Zhang, and Z. Zhang, Eds., *Conformer: Convolution-augmented Transformer for Speech Recognition*, 2020.
- [26] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An asr corpus based on public domain audio books,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015, pp. 5206–5210.
- [27] T. Kudo and J. Richardson, “Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” 2018. [Online]. Available: <https://arxiv.org/abs/1808.06226>
- [28] J. Park, Kyubyong Kim, “g2pe,” <https://github.com/Kyubyong/g2pe>, 2019.